Truthy vs Falsy

The 8 falsy values, and when ?? beats ||

FALSY `false` `0` `-0` `0n` `""` `null` `undefined` `NaN`

TRUTHY `"0"` `"false"` `[]` `{}` `-1` `Infinity` + all the rest

	a || 10	a ?? 10
a = 0	10	0
a = ""	10	""
a = null	10	10
a = undefined	10	10

1 && and || return an operand

```
"" || "x"          // "x"
1 && "a"           // "a"
0 && "a"           // 0
a ?? b || c        // SyntaxError
```

2 ?. guards one step only

```
const user = { address: null };
user.address?.city  // undefined
user?.address.city  // TypeError
user.run?.()        // undefined
```

TRAP What does this log?

```
if ([]) console.log("truthy");
console.log([] == false);
```

truthy

true

if () checks truthiness, == coerces types.

KEY TAKEAWAY

`??` Default for null or undefined

`?.` Guards one step only

`||` Falls back on any falsy

? Follow-up questions to expect

- **Is an empty array truthy?** → Yes. Every object is truthy, even [] and new Boolean(false).
- `null` **vs** `undefined`**?** → undefined = never set, null = empty on purpose.
- **Can** `??` **mix with** `||` **or** `&&`**?** → Not without ( ): it is a SyntaxError.
- **Does** `?.` **skip the whole chain?** → If its left side is nullish. Later steps need ?. too.